Red Hat OpenShift AI Self-Managed 3.4

Working with AutoRAG

Use AutoRAG in Red Hat OpenShift AI Self-Managed

Last Updated: 2026-06-09

Red Hat OpenShift AI Self-Managed 3.4 Working with AutoRAG

Use AutoRAG in Red Hat OpenShift AI Self-Managed

Legal Notice

Abstract

Use AutoRAG in Red Hat OpenShift AI Self-Managed to automatically optimize and evaluate retrieval-augmented generation (RAG) configurations for your documents and use case.

Table of Contents

PREFACE

You can use AutoRAG in OpenShift AI to optimize and evaluate retrieval-augmented generation (RAG) configurations for your documents and use cases.

CHAPTER 1. AUTORAG OVERVIEW

AutoRAG is an automated optimization system in Red Hat OpenShift AI that finds the best retrieval-augmented generation (RAG) configuration for your documents and use case. You provide documents and test data. AutoRAG tests different RAG configurations, ranks results by evaluation metrics, and generates notebooks to run RAG patterns.

1.1. AUTORAG WORKFLOW

When you create an AutoRAG optimization run, AutoRAG tests combinations of chunking, embedding, retrieval, and generation settings against your test data. Each combination produces a RAG pattern with evaluation scores. AutoRAG ranks patterns on a leaderboard and generates Jupyter notebooks that you can use to run the best pattern.

AutoRAG automatically samples up to 1 GiB of relevant documents based on your test data, so you do not need to filter your document set in advance. Documents referenced in your test data are prioritized during sampling.

1.2. AUTORAG TERMINOLOGY

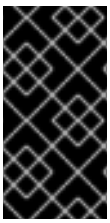
RAG pattern

An optimized RAG configuration that includes performance metrics, a leaderboard position, and indexing and inference notebooks.

Search space

The set of parameter combinations that AutoRAG tests during optimization. The search space includes chunking, embedding, retrieval, and generation settings.

1.3. TECHNOLOGY PREVIEW LIMITATIONS



IMPORTANT

AutoRAG is a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. For more information, see Technology Preview Features Support Scope .

The following limitations apply during Technology Preview:

- Only English language documents are supported.
- Remote Milvus vector databases only. Inline Milvus is not supported.
- A maximum of three foundation models and two embedding models per optimization run. Specifying more models can cause the run to fail.
- Images embedded in documents are not processed.
- Optical character recognition (OCR) is not available for PDF documents.
- Table structure detection is not available for PDF documents.

1.4. VIEWING EXTERNALLY CREATED RUNS

If you import an AutoRAG pipeline to your pipeline server and create runs from it, the runs appear on the AutoRAG page in the dashboard. When you import the pipeline, set the pipeline name to **documents-rag-optimization-pipeline** and the pipeline version name to **documents-rag-optimization-pipeline-<version>**, such as **documents-rag-optimization-pipeline-3.4.0**.

For more information about importing pipelines, see Importing a pipeline.

Additional resources

- Prepare test data for AutoRAG
- Create an AutoRAG optimization run
- AutoRAG evaluation metrics
- AutoRAG configuration parameters

To find the best RAG configuration for your documents, prepare test data, create an AutoRAG optimization run, evaluate the results, and run the best-performing pattern.

Before you begin, ensure that the following prerequisites are met:

- A Llama Stack instance is available and configured with foundation and embedding models. For more information, see Working with Llama Stack.
- A remote Milvus vector database is registered with your Llama Stack instance. Inline Milvus is not supported.
- Your documents are available in an S3-compatible storage bucket or locally for upload.

IMPORTANT

Upload the updated AutoRAG pipeline definition before you create your first run. For instructions and download links, see RHOAIENG-64768 - AutoML and AutoRAG pipeline runs fail with image pull errors in the release notes.

CHAPTER 2. PREPARE TEST DATA FOR AUTORAG

Create a test data file that AutoRAG uses to evaluate RAG configurations against ground-truth answers. The test data contains questions, expected answers, and the document IDs where those answers can be found.

Your test data file must be a JSON array with the following structure:

```
[
  {
    "question": "What is the return policy?",
    "correct_answers": ["Items can be returned within 30 days", "The return window is 30 days from purchase"],
    "correct_answer_document_ids": ["policies.pdf", "faq.md"]
  },
  {
    "question": "How do I reset my password?",
    "correct_answers": ["Navigate to Settings and select Reset Password"],
    "correct_answer_document_ids": ["user-guide.pdf"]
  }
]
```

The test data file uses the following fields:

question

The question that AutoRAG submits to each RAG configuration during evaluation.

correct_answers

One or more expected answers. Multiple phrasings help AutoRAG evaluate answer correctness more accurately.

correct_answer_document_ids

The file names of documents that contain the answer. AutoRAG uses these IDs to prioritize documents during sampling and to evaluate context correctness.

Prerequisites

- You have access to the documents that you plan to use with AutoRAG.
- You have a text editor or scripting environment for creating JSON files.

Procedure

1. Identify questions that represent your target use case.
   Select questions that cover the range of topics in your document set. Include both simple factual questions and questions that require synthesizing information from multiple sections.
2. For each question, write one or more correct answers.
   Use the exact phrasing from your source documents where possible. Include alternative phrasings to improve evaluation accuracy.
3. For each question, identify the source documents that contain the answer.
   Use the base file name without any folder path. For example, use **policies.pdf**, not **documents/policies.pdf**.

4. Save your test data as a JSON file.
   Ensure the file uses the **.json** extension and follows the JSON format shown earlier in this procedure.

5. Make the test data file available for your optimization run:

   - Upload the file to an S3-compatible storage bucket.
   - Keep the file on your local system to upload when you create the optimization run.

Verification

- Open the JSON file in a text editor and verify that it is valid JSON.
- Verify that each entry contains the **question** and **correct_answers** fields.
- Verify that the document IDs in **correct_answer_document_ids** match the file names in your document set.

Additional resources

- Create an AutoRAG optimization run
- AutoRAG evaluation metrics

CHAPTER 3. CREATE AN AUTORAG OPTIMIZATION RUN

Create an AutoRAG optimization run to find the best RAG configuration for your documents and use case. You configure the optimization run in a two-step wizard that collects connection details and optimization settings.

Prerequisites

- You have editor access to a project in OpenShift AI.
- Gen AI studio is enabled in the OpenShift AI dashboard.
- A Llama Stack instance is available and configured with foundation and embedding models. For more information, see Working with Llama Stack.
- A remote Milvus vector database is registered with your Llama Stack instance. Inline Milvus is not supported.
- A Llama Stack connection is configured in your project. The connection must include the Llama Stack base URL and API key.
- Your documents are available in an S3-compatible storage bucket or locally for upload.
- If you store multiple documents in S3, they are in a single folder in the bucket.
- Documents are in one of the following formats: PDF, DOCX, PPTX, Markdown, HTML, or TXT.
- You have prepared a test data file in JSON format. For more information, see Prepare test data for AutoRAG.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio** > **AutoRAG**.
2. Select your project, and then click **Create AutoRAG optimization run**.
3. Enter a name and optionally a description for the optimization run, select a **Llama Stack connection**, and click **Next**.
4. Configure the optimization settings as follows:
   a. In the **Knowledge setup** section, select an S3 connection for your documents and select the files to use. You can browse your S3 bucket or upload files directly. Uploaded files can be up to 32 MiB.
   b. Select your Milvus vector database from the **Vector I/O provider** list.
   c. Add an evaluation data set by browsing your S3 bucket for a JSON file or by uploading one directly. You can download a template from the **evaluation dataset template** link on the configuration page.
   d. Select an optimization metric from the **Optimization metric** list:
      - **Answer faithfulness**: Optimizes for answers grounded in retrieved context.
      - **Answer correctness**: Optimizes for answers that match your test data.

- **Context correctness**: Optimizes for retrieval of relevant documents.

e. In the **Maximum RAG patterns** field, enter a value between 4 and 20. The default is 8.

f. Optional: Click **Edit** on the model configuration card to exclude models. By default, all foundation and embedding models available from your Llama Stack instance are selected. Select no more than 3 foundation models and 2 embedding models to avoid run failures.

TIP

For embedding models, **BAAI/bge-m3** is recommended. For more information, see AutoRAG configuration parameters.

NOTE

Optimization runs cannot be edited after creation. To stop, archive, or delete the underlying pipeline run, see Managing pipeline runs.

5. Click **Create run**.
AutoRAG begins testing RAG configurations. You can monitor the optimization run status on the AutoRAG page.

Verification

- On the **AutoRAG** page, the new optimization run is listed with a status of **Running** or **Pending**.
- The run progresses to **Complete** when AutoRAG finishes testing all RAG configurations.

Additional resources

- Evaluate AutoRAG results
- AutoRAG configuration parameters
- AutoRAG evaluation metrics

CHAPTER 4. EVALUATE AUTORAG RESULTS

After an AutoRAG optimization run completes, review the leaderboard and pattern details to select the best RAG configuration for your use case.

Prerequisites

- You have created an AutoRAG optimization run and it has completed successfully.

Procedure

1. In the OpenShift AI dashboard, click **Gen AI studio** > **AutoRAG**.
2. Select a project from the project list.
3. Click the name of the completed optimization run.
4. Review the leaderboard, which ranks RAG patterns by your selected optimization metric. Compare scores across all metric columns to evaluate each pattern holistically. The score columns show mean values for each metric, and an indicator marks the optimization metric. For information about how each metric is calculated, see AutoRAG evaluation metrics.
5. To view detailed information about a pattern, click the pattern name or select **View details** from the actions menu.
   The pattern details view shows:
   - Scores for each metric with mean, confidence interval high, and confidence interval low values
   - Configuration settings organized by category: chunking, embedding, retrieval, and generation
   - **Sample Q&A** results with per-question scores, your expected answers, and the pattern's generated answers
6. Compare patterns by examining **Sample Q&A** responses across different patterns.
   Review answers in the **Sample Q&A** tab to verify that patterns produce accurate, well-grounded responses.
7. After you choose a pattern, save the notebooks for the pattern by doing the following from the actions menu:
   a. Select **Save as indexing notebook**
   b. Select **Save as inference notebook**

Verification

- You have selected a RAG pattern based on its leaderboard scores and Sample Q&A results.
- The indexing notebook and inference notebook for the selected pattern are downloaded to your system.

Additional resources

- Run the RAG pattern

- AutoRAG evaluation metrics

CHAPTER 5. RUN THE RAG PATTERN

After you select a RAG pattern from the AutoRAG leaderboard, run the notebooks in an OpenShift AI workbench to use the pattern with your documents.

Prerequisites

- You have selected a RAG pattern from a completed AutoRAG optimization run. For more information, see Evaluate AutoRAG results.
- You have downloaded the indexing and inference notebooks from the AutoRAG leaderboard or pattern details view.
- You have a running workbench in your OpenShift AI project.
- You have the connection details for your S3-compatible object storage and your Llama Stack instance.
- The models used in your selected RAG pattern are available on your Llama Stack instance.

Procedure

1. In the OpenShift AI dashboard, open your workbench.
2. Attach the following data connections to the workbench. Use the same S3 bucket and Llama Stack instance that you used for the optimization run.
    - An S3-compatible object storage connection for your documents
    - A connection for your Llama Stack instance that includes the API key and base URL
3. Optional: To index additional documents, upload and run the indexing notebook.
   The vector database is already populated from the optimization run.
4. Upload the inference notebook to the workbench.
5. Open the inference notebook and run each cell.
   The notebook prompts you to enter a question. Enter a question to verify that the RAG system returns relevant answers from your documents.

Verification

- If you ran the indexing notebook, verify that all cells completed without errors.
- The inference notebook returns answers grounded in your documents.
- If the notebook displays an error, check your connection details.

Additional resources

- Evaluate AutoRAG results

CHAPTER 6. AUTORAG EVALUATION METRICS

Each AutoRAG optimization run produces scores for three evaluation metrics. All metrics are scored from 0 to 1, with higher scores indicating better performance. Each score includes a mean value, a high confidence interval (CI high), and a low confidence interval (CI low).

Answer faithfulness

Measures whether the generated answer uses information from the retrieved context rather than hallucinated content. A high faithfulness score means the answer uses information from the retrieved documents, not from the model's training data.
Optimize for faithfulness when accuracy and traceability to source documents are critical, such as in compliance or legal use cases.

Answer correctness

Measures whether the generated answer matches the expected ground-truth answers in your test data. A high answer correctness score means the RAG system produces answers that align with your provided correct answers.
Optimize for answer correctness when you have well-defined expected answers and want the system to match them closely.

Context correctness

Measures whether the retrieved documents are relevant to the question. A high context correctness score means the retrieval step finds the relevant documents from your document set.
Optimize for context correctness when retrieval quality is more important than answer phrasing, such as when you plan to use a different generation model in production.

6.1. METRIC COMBINATIONS

When you review AutoRAG results, consider metric scores together:

- High faithfulness with low answer correctness indicates that the answer is grounded in the retrieved context, but the context does not contain the most accurate answer.
- High answer correctness with low faithfulness indicates that the model draws on training data rather than retrieved documents.
- Low context correctness with high answer correctness indicates that the generation model is compensating for poor retrieval. This pattern might not be reliable for questions outside your test data.

Additional resources

- Evaluate AutoRAG results
- AutoRAG configuration parameters

CHAPTER 7. AUTORAG CONFIGURATION PARAMETERS

The following user-configurable parameters are available when you create an AutoRAG optimization run in the OpenShift AI dashboard. AutoRAG also uses default values for search space parameters that you cannot change.

7.1. USER-CONFIGURABLE PARAMETERS

You set the following parameters when you create an AutoRAG optimization run.

Table 7.1. User-configurable parameters

Parameter	Description	Values
Optimization metric	The metric that AutoRAG uses to rank RAG patterns.	Answer faithfulness (default), Answer correctness, Context correctness
Maximum RAG patterns	The number of RAG configurations that AutoRAG evaluates.	A value between 4 and 20. Default: 8.
Foundation models	The large language models used for answer generation. Discovered from your Llama Stack instance.	All available models are selected by default. Clear the checkbox for models to exclude them.
Embedding models	The models used to create vector embeddings and to encode queries during retrieval. Discovered from your Llama Stack instance.	All available models are selected by default. Clear the checkbox for models to exclude them.
Vector database	The Milvus instance where AutoRAG stores document embeddings. Inline Milvus is not supported.	A remote Milvus instance registered with your Llama Stack instance.
Input documents	Documents that AutoRAG processes and indexes for retrieval.	PDF, DOCX, PPTX, Markdown, HTML, TXT. Maximum 32 MiB per file when uploading. Documents in S3 can be selected via the file browser without upload size restrictions.
Evaluation dataset	A JSON file with test questions and expected answers for evaluating RAG quality.	JSON format. See Prepare test data for AutoRAG.

7.2. SEARCH SPACE DEFAULTS

AutoRAG explores the following search space during optimization. For parameters with multiple default values, AutoRAG evaluates combinations of those values across RAG configurations. These values are not configurable through the dashboard.

Table 7.2. Search space default parameters

Parameter	Default values	Description
Chunking method	Recursive (recursive character text splitting)	The method used to split documents into chunks.
Chunk size	1024, 2048	The target size of each document chunk in characters.
Chunk overlap	128, 256	The number of overlapping characters between consecutive chunks.
Retrieval method	Simple (direct chunk retrieval)	The method used to retrieve relevant chunks from the vector database.
Number of chunks	3, 5, 10	The number of document chunks retrieved per query.
Search mode	Vector, Hybrid	The search strategy. Hybrid search combines vector and keyword search and is available only with Milvus.

7.3. RECOMMENDED EMBEDDING MODELS

For best results, use **BAAI/bge-m3** as your embedding model. It supports more than 100 languages, dense and sparse retrieval, and requires approximately 1.1 GB of memory (fp16).

Additional resources

- Create an AutoRAG optimization run
- AutoRAG evaluation metrics